

Q.No:7

Problem Description:

Any sequence A of size n is called **B-sequence** if:

$A_1 < A_2 < \dots < A_k > A_{k+1} > A_{k+2} > \dots > A_n$ where $1 \leq k \leq n$. That is, a sequence which is initially strictly increasing and then strictly decreasing (the decreasing part may or may not be there).

All elements in A except the maximum element comes at most twice (once in increasing part and once in decreasing part) and maximum element comes exactly once.

All elements coming in decreasing part of sequence should have come once in the increasing part of sequence.

You are given a B-sequence S and Q operations. For each operation, you are given a value **val**. You have to insert **val** in S if and only if after insertion, S still remains a B-sequence.

After each operation, print the size of S . After all the operations, print the sequence S .

Hint: Think of using some data structure to support insertion of elements in complexity better than linear.

Input Constraints:

$$1 \leq N \leq 10^5$$

$$1 \leq S_i \leq 10^9$$

$$1 \leq Q \leq 10^5$$

$$1 \leq val \leq 10^9$$

Given sequence S is a **B-sequence**,

Input Format:

First line consists of an integer N , denoting size of S .

Second line consists of N space separated integers, denoting elements of S .

Next line consists of an integer Q , denoting number of operations.

Each of the following Q lines consists of an integer **val**.

Output Format:

After each operation, print the size of S in a new line.

After all operations, print the sequence S .

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
4
1 2 5 2
4
5
1
3
2
```

EXPECTED OUTPUT

```
4
5
6
6
1 2 3 5 2 1
```

Test Case 2

INPUT (STDIN)

```
5
3 4 5 1 3
3
6
2
3
1
```

EXPECTED OUTPUT

```
6
7
7
1 2 3 4 5 6 3
```

Mandatory Test Cases

Code:

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 200005

long long a[MAX];
long long queries[MAX];
long long values[MAX];
int cnt[MAX];

int compare(const void *p1, const void *p2)
{
    long long x = *(long long *)p1;
```

```
long long y = *(long long *)p2;

if (x < y)
return -1;
if (x > y)
return 1;
return 0;
}

int lowerBound(long long x, int n)
{
int l = 0;
int r = n - 1;

while (l <= r)
{
int mid = (l + r) / 2;

if (values[mid] == x)
return mid;

if (values[mid] < x)
l = mid + 1;
else
r = mid - 1;
}

return -1;
}

int main()
{
int n, q;

scanf("%d", &n);

for (int i = 0; i < n; i++)
scanf("%lld", &a[i]);

scanf("%d", &q);

for (int i = 0; i < q; i++)
scanf("%lld", &queries[i]);

int total = n + q;
long long temp[MAX];

for (int i = 0; i < n; i++)
temp[i] = a[i];

for (int i = 0; i < q; i++)
temp[n + i] = queries[i];
```

```
qsort(temp, total, sizeof(long long), compare);

int m = 0;

for (int i = 0; i < total; i++)
{
    if (i == 0 || temp[i] != temp[i - 1])
        values[m++] = temp[i];
}

for (int i = 0; i < n; i++)
{
    int pos = lowerBound(a[i], m);
    cnt[pos]++;
}

long long maximum = values[m - 1];
int size = n;

for (int i = 0; i < q; i++)
{
    long long x = queries[i];
    int pos = lowerBound(x, m);

    if (cnt[pos] == 0)
    {
        cnt[pos]++;
        size++;
    }

    if (x > maximum)
        maximum = x;
}

else if (cnt[pos] == 1 && x != maximum)
{
    cnt[pos]++;
    size++;
}

printf("%d\n", size);
}

int first = 1;

for (int i = 0; i < m; i++)
{
    if (cnt[i] > 0)
    {
        if (!first)
            printf(" ");

        printf("%lld", values[i]);
        first = 0;
    }
}
```

```
}  
}  
  
for (int i = m - 1; i >= 0; i--)  
{  
if (cnt[i] == 2 && values[i] != maximum)  
printf(" %lld", values[i]);  
}  
  
printf("\n");  
  
return 0;  
}
```

Output:

Test 1:

```
aadityajaiswal@Aadityas-MacBook-Air:~$  
4  
1 2 5 2  
4  
1  
3  
2  
5  
5  
6  
6  
6  
1 2 3 5 2 1  
aadityajaiswal@Aadityas-MacBook-Air:~$
```

Test 2:

```
aadityajaiswal@Aadityas-MacBook-Air:~$  
5  
3 4 5 1 3  
3  
6  
2  
3  
6  
7  
7  
1 2 3 4 5 6 3  
aadityajaiswal@Aadityas-MacBook-Air:~$
```